SQL - Day - 13 Hands On - Sana Naveen

Problem 1

Problem Statement :

Pre-Requisites: Before starting with problem solving, please make sure that you have created a database and restored data (refer Sample Database create objects.txt and load data.txt)
Level-1: Problem 1 - Basic Customer Order Report
Scenario:
The store manager wants a simple report showing customer orders along with their order dates and status. This report will help track pending and completed orders.
Requirements
1. Retrieve customer first name, last name, order_id, order_date, and order_status.
2. Display only orders with status Pending (1) or Completed (4).
3. Sort the results by order_date in descending order.

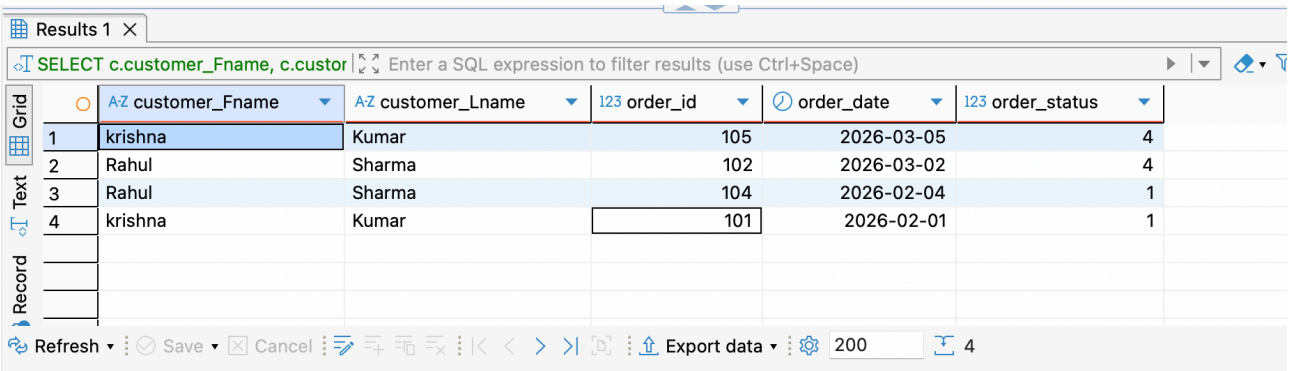
Technical Constraints
- Use SELECT statement.
- Use WHERE clause with logical operators (AND/OR).
- Use ORDER BY clause.
- Use INNER JOIN between customers and orders tables.
Expectations:
Students should write a correct query using joins and filters, and properly order the result set.
Learning Outcome
Understand basic SELECT queries, filtering using WHERE conditions, logical operators, and sorting using ORDER BY clause with INNER JOIN.

Code :

```
CREATE DATABASE store;

use store


CREATE TABLE customer (
   customer_id INT PRIMARY KEY,
   customer_Fname VARCHAR(20) NOT NULL,
   customer_Lname VARCHAR(20) NOT NULL
);

CREATE TABLE orders (
   order_id INT PRIMARY KEY,
   order_date DATE NOT NULL,
   order_status INT CHECK (order_status IN (1,2,3,4)),
   c_id INT,
   FOREIGN KEY (c_id) REFERENCES customer(customer_id)
);

INSERT INTO customer VALUES
(1,'krishna','Kumar'),
(2,'Rahul','Sharma'),
(3,'Vijay','Verma'),
```

```
(4,'Sneha','Reddy');

INSERT INTO orders VALUES
(101,'2026-02-01',1,1),
(102,'2026-03-02',4,2),
(103,'2026-03-03',2,3),
(104,'2026-02-04',1,2),
(105,'2026-03-05',4,1);

SELECT
   c.customer_Fname,
   c.customer_Lname,
   o.order_id,
   o.order_date,
   o.order_status
FROM customer c
INNER JOIN orders o
ON c.customer_id = o.c_id
WHERE o.order_status = 1
  OR o.order_status = 4
ORDER BY o.order_date DESC;
```

Output Screenshot :



Explanation :

The clause FROM customer c INNER JOIN orders o ON c.customer_id = o.c_id is used to combine data from the customer and orders tables based on a related column. c and o are table aliases that provide short names for the tables. The INNER JOIN ensures that only rows with matching values between customer.customer_id and orders.c_id are returned. This allows the query to retrieve customer information along with their corresponding order details in a single result set.

Problem 2

Problem Statement :
Product Price Listing by Category
Scenario:
The sales team wants a product listing categorized by product category along with brand details to understand product distribution.
Requirements
1. Display product_name, brand_name, category_name, model_year, and list_price.
2. Filter products with list_price greater than 500.
3. Sort results by list_price in ascending order.

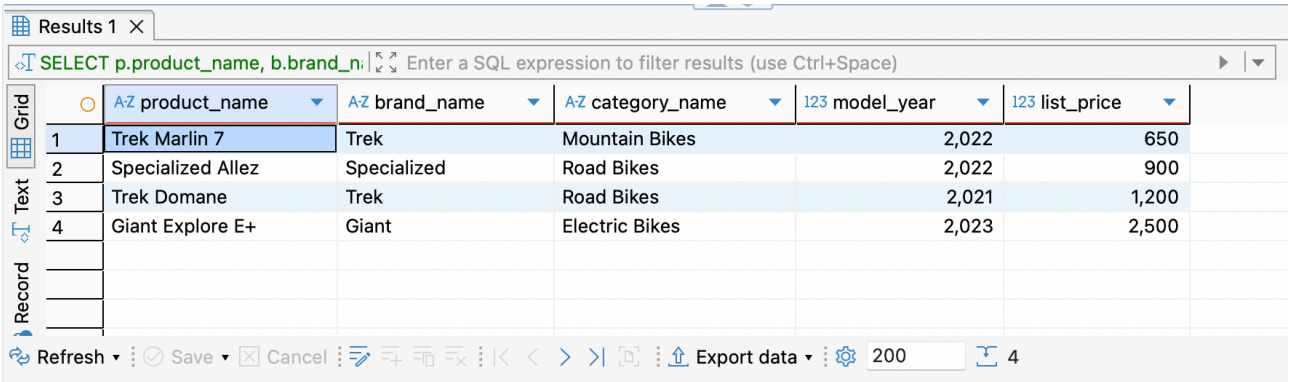
Technical Constraints
- Use SELECT statement.
- Use WHERE clause.
- Use ORDER BY clause.
- Use INNER JOIN between products, brands, and categories tables.

Expectations:
Students should correctly join multiple tables and apply filtering and sorting logic.
Learning Outcome
Learn multi-table joins, filtering numeric conditions, and sorting query results.

Code :
```
CREATE DATABASE cart;
GO

USE cart;
GO

CREATE TABLE categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL
);

CREATE TABLE brands (
    brand_id INT PRIMARY KEY,
    brand_name VARCHAR(50) NOT NULL
);

CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    brand_id INT,
    category_id INT,
    model_year INT,
    list_price DECIMAL(10,2),
    FOREIGN KEY (brand_id) REFERENCES brands(brand_id),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
INSERT INTO categories VALUES
(1,'Mountain Bikes'),
(2,'Road Bikes'),
(3,'Electric Bikes');
```

```
INSERT INTO brands VALUES
(1,'Trek'),
(2,'Giant'),
(3,'Specialized');

INSERT INTO products VALUES
(101,'Trek Marlin 7',1,1,2022,650),
(102,'Giant Talon 3',2,1,2023,480),
(103,'Specialized Allez',3,2,2022,900),
(104,'Trek Domane',1,2,2021,1200),
(105,'Giant Explore E+',2,3,2023,2500);

SELECT
    p.product_name,
    b.brand_name,
    c.category_name,
    p.model_year,
    p.list_price
FROM products p
INNER JOIN brands b
    ON p.brand_id = b.brand_id
INNER JOIN categories c
    ON p.category_id = c.category_id
WHERE p.list_price > 500
ORDER BY p.list_price ASC;
```

Output Screenshot :



	product_name	brand_name	category_name	model_year	list_price
1	Trek Marlin 7	Trek	Mountain Bikes	2,022	650
2	Specialized Allez	Specialized	Road Bikes	2,022	900
3	Trek Domane	Trek	Road Bikes	2,021	1,200
4	Giant Explore E+	Giant	Electric Bikes	2,023	2,500

Explanation :
This query retrieves product information by joining the products, brands, and categories tables using INNER JOIN to combine related data through foreign key relationships. It selects the product name, brand name, category name, model year, and list price. The WHERE clause filters products with a list_price greater than 500, and the ORDER BY clause sorts the results in ascending order based on the product price. This demonstrates multi-table joins, numeric filtering, and sorting of query results.

Problem 3

Problem Statement :

Store Wise Sales Summary

Scenario:

The regional manager wants to analyze total sales generated by each store.
Requirements
1. Display store_name and total sales amount.
2. Calculate total sales using (quantity * list_price * (1 - discount)).
3. Include only completed orders (order_status = 4).
4. Group results by store_name.
5. Sort total sales in descending order.

Technical Constraints
- Use INNER JOIN between stores, orders, and order_items tables.
- Use GROUP BY clause.
- Use aggregate functions such as SUM().
- Use WHERE clause for filtering.
- Use ORDER BY clause.

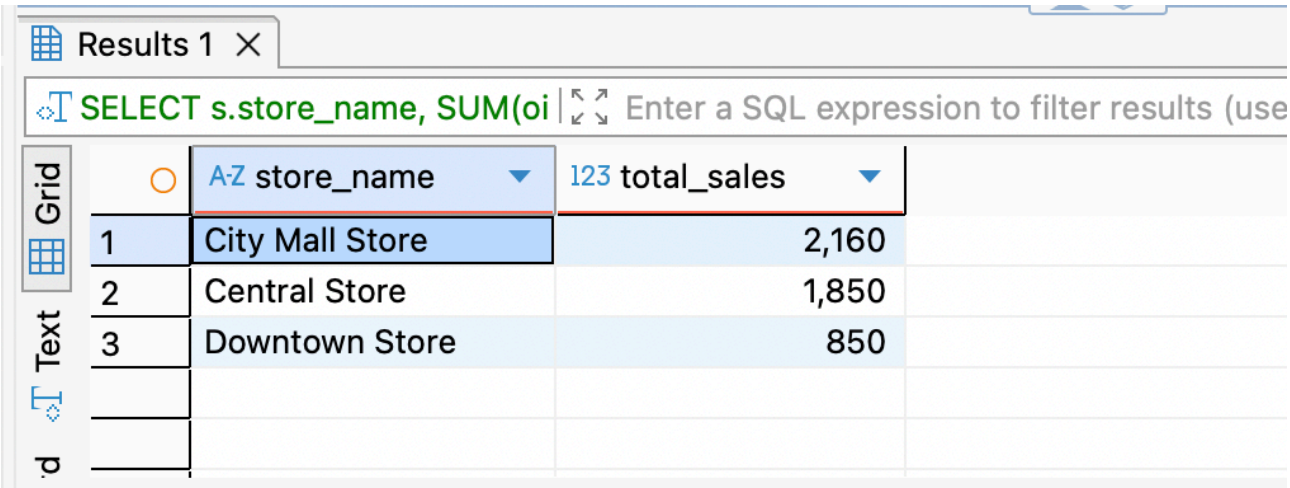
Expectations:
Students must write a grouped query with calculated columns and apply proper filtering.
Learning Outcome
Understand GROUP BY, aggregate functions, calculated fields, and filtering grouped data.

Code :

```
CREATE DATABASE cart;
GO

USE cart;
GO

CREATE TABLE stores (
    store_id INT PRIMARY KEY,
    store_name VARCHAR(50) NOT NULL
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    order_date DATE NOT NULL,
    order_status INT NOT NULL,
    store_id INT,
    FOREIGN KEY (store_id) REFERENCES stores(store_id)
);

CREATE TABLE order_items (
    item_id INT PRIMARY KEY,
    order_id INT,
    quantity INT NOT NULL,
    list_price DECIMAL(10,2) NOT NULL,
    discount DECIMAL(4,2) DEFAULT 0,
```

```
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

INSERT INTO stores VALUES
(1,'Central Store'),
(2,'City Mall Store'),
(3,'Downtown Store');

INSERT INTO orders VALUES
(101,'2026-03-01',4,1),
(102,'2026-03-02',4,2),
(103,'2026-03-03',1,1),
(104,'2026-03-04',4,3),
(105,'2026-03-05',4,2);

INSERT INTO order_items VALUES
(1,101,2,500,0.10),
(2,101,1,1000,0.05),
(3,102,3,300,0.00),
(4,104,5,200,0.15),
(5,105,2,700,0.10);

SELECT
    s.store_name,
    SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS total_sales
FROM stores s
INNER JOIN orders o
    ON s.store_id = o.store_id
INNER JOIN order_items oi
    ON o.order_id = oi.order_id
WHERE o.order_status = 4
GROUP BY s.store_name
ORDER BY total_sales DESC;
```

Output Screenshot :



Explanation :

This query calculates the total sales generated by each store by joining the stores, orders, and order_items tables using INNER JOIN based on their related keys. The WHERE clause filters only completed orders (order_status = 4). The sales amount is calculated using the formula (quantity * list_price * (1 - discount)), and the SUM() aggregate function computes the total sales for each store. The results are grouped using GROUP BY store_name and sorted in descending order of total sales using ORDER BY.

Problem 4

Problem Statement :
Product Stock and Sales Analysis
Scenario:
The inventory manager wants to compare stock availability and total quantity sold for each product.
Requirements
1. Display product_name, store_name, available stock quantity, and total quantity sold.
2. Include products even if they have not been sold (use appropriate join).
3. Group results by product_name and store_name.
4. Sort results by product_name.
Technical Constraints
- Use LEFT JOIN between stocks and order_items.
- Use INNER JOIN with products and stores tables.
- Use GROUP BY clause.
- Use SUM() aggregate function.
- Use ORDER BY clause.
Expectations:
Students should correctly use different join types (INNER and LEFT), grouping, and aggregate functions.
Learning Outcome
Develop strong understanding of different JOIN types (INNER, LEFT), aggregation, grouping data, and analyzing relational datasets.

Code :
```
CREATE DATABASE InventoryDB;
GO

USE InventoryDB;
GO


CREATE TABLE products (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(50) NOT NULL
);

CREATE TABLE stores (
   store_id INT PRIMARY KEY,
   store_name VARCHAR(50) NOT NULL
);

CREATE TABLE stocks (
   store_id INT,
   product_id INT,
   quantity INT,
   PRIMARY KEY (store_id, product_id),
   FOREIGN KEY (store_id) REFERENCES stores(store_id),
   FOREIGN KEY (product_id) REFERENCES products(product_id)
);

CREATE TABLE orders (
   order_id INT PRIMARY KEY,
```

```sql
    order_date DATE
);

CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

INSERT INTO products VALUES
(1,'Laptop'),
(2,'Mobile'),
(3,'Headphones');

INSERT INTO stores VALUES
(1,'Central Store'),
(2,'City Mall Store');

INSERT INTO stocks VALUES
(1,1,50),
(1,2,100),
(2,1,30),
(2,3,70);

INSERT INTO orders VALUES
(101,'2026-03-01'),
(102,'2026-03-02');

INSERT INTO order_items VALUES
(1,101,1,5),
(2,101,2,10),
(3,102,1,3);

SELECT
    p.product_name,
    s.store_name,
    st.quantity AS stock_quantity,
    SUM(oi.quantity) AS total_quantity_sold
FROM stocks st
INNER JOIN products p
    ON st.product_id = p.product_id
INNER JOIN stores s
    ON st.store_id = s.store_id
LEFT JOIN order_items oi
    ON p.product_id = oi.product_id
GROUP BY
    p.product_name,
    s.store_name,
    st.quantity
ORDER BY
    p.product_name;
```

Output screenshot :

Results 1

SELECT p.product_name, s.store_na | Enter a SQL expression to filter results (use Ctrl+Space)

	product_name	store_name	stock_quantity	total_quantity_sold
1	Headphones	City Mall Store	70	[NULL]
2	Laptop	Central Store	50	8
3	Laptop	City Mall Store	30	8
4	Mobile	Central Store	100	10

Explanation :

This query analyzes product stock and sales by joining the stocks, products, stores, and order_items tables. INNER JOIN is used to connect products and stores through the stocks table, while LEFT JOIN ensures that products with no sales are still included in the result. The SUM() aggregate function calculates the total quantity sold for each product, and GROUP BY organizes the results by product name and store name. Finally, the results are sorted using ORDER BY product_name.